WHERE THEY STAND

Repository Build Handoff

Instructions for the coding agent to transform the existing GitHub repository into the working application foundation without changing the project's political methodology.

Current repository state	Assume this already exists
README.md	Already edited by project owner.
docs/	Already created.
Initial technical handoff PDF	Already uploaded inside docs/. Do not replace or delete it.

Primary instruction: Start from the repository as it exists. Preserve the README and initial handoff. Add the application foundation around them. Do not ask the owner to recreate repository setup that is already complete.

1. Coding agent mission

Turn the existing repository into a runnable, testable monorepo foundation for Where They Stand. This handoff is about repository changes and first implementation scaffolding. The larger product behavior, neutrality methodology, research system, and data architecture are defined in the initial technical handoff already stored in docs/.

The coding agent should:

• Inspect the existing repository before making changes.

• Read README.md and every file currently in docs/ before coding.

• Preserve existing project documentation.

• Create the monorepo, web application, shared packages, Python worker skeleton, local infrastructure, CI, and documentation placeholders described below.

• Make the repository runnable locally after the first implementation pass.

• Do not begin candidate scraping, political stance classification, or AI publication logic in this repository-setup pass.

• Commit changes in logical units and keep main buildable.

2. Repository end state

After this handoff is implemented, the repository should have this shape:

```
where-they-stand/ .github/ workflows/ ci.yml apps/ web/ worker/ packages/ contracts/ db/
issue-definitions/ ui/ services/ collectors/ congress/ fec/ google-civic/ meta/ web/ x/ youtube/ bluesky/
classification/ extraction/ source-archive/ infra/ docker/ docker-compose.yml deploy/ monitoring/ docs/
[PRESERVE EXISTING INITIAL HANDOFF PDF] methodology/ editorial/ api/ runbooks/ tests/ e2e/ fixtures/
neutrality/ .editorconfig .env.example .gitignore package.json pnpm-workspace.yaml turbo.json README.md
```

3. Files that must be preserved

Path	Instruction
README.md	Do not overwrite wholesale. Preserve the owner's existing copy. Add setup instructions only if needed.
docs/* existing files	Do not rename, replace, or delete the initial technical handoff.
Git history	Do not squash or rewrite existing history.
main branch	Do not force-push. Use a feature branch and pull request when the connected environment supports it.

4. Root workspace changes

Use pnpm workspaces and Turborepo. The root package should coordinate JavaScript and TypeScript workspaces. Python remains independently managed inside apps/worker.

Create pnpm-workspace.yaml

```
packages: - "apps/*" - "packages/*" - "services/*" - "services/collectors/*"
```

Create root package.json

```
{ "name": "where-they-stand", "private": true, "scripts": { "dev": "turbo dev", "build": "turbo build",
"lint": "turbo lint", "typecheck": "turbo typecheck", "test": "turbo test" }, "devDependencies": {
```

```
"turbo": "^2" }, "packageManager": "pnpm@10" }
```

Use the current stable pnpm 10.x release when the repository is initialized. Commit the generated lockfile.

Create turbo.json

```
{ "$schema": "https://turbo.build/schema.json", "tasks": { "dev": { "cache": false, "persistent": true },
"build": { "dependsOn": ["^build"], "outputs": [".next/**", "dist/**"] }, "lint": { "dependsOn":
["^lint"] }, "typecheck": { "dependsOn": ["^typecheck"] }, "test": { "dependsOn": ["^build"], "outputs":
["coverage/**"] } } }
```

5. Web application

Create apps/web as a Next.js App Router application using TypeScript, ESLint, Tailwind CSS, src/, and the @/* alias. Do not build the finished aesthetic yet. Establish the design tokens and basic shell so later coding has a stable base.

• App Router.

• TypeScript strict mode.

• Tailwind CSS.

• Responsive layout.

• Server components by default.

• No authentication implementation yet beyond an admin placeholder.

• No political red/blue design language.

• Neutral base palette: warm white/light gray, charcoal/deep slate, muted teal accent.

• Create a simple homepage proving the app runs. It should identify Where They Stand and state that the platform does not endorse candidates.

Create route placeholders:

```
apps/web/src/app/ page.tsx find/page.tsx issues/page.tsx issue/[slug]/page.tsx races/[raceId]/page.tsx
candidate/[candidateId]/page.tsx compare/[raceId]/page.tsx methodology/page.tsx corrections/page.tsx
about/page.tsx admin/page.tsx
```

6. Shared packages

Package	First-pass responsibility
packages/contracts	Shared TypeScript/Zod contracts. Define stance enums, candidate/race IDs, API result shells.
packages/db	Database package, Prisma schema or equivalent ORM configuration, client export, migrations directory.
packages/issue-definitions	Versioned machine-readable definitions for all 15 issues. This is a protected policy-definition layer.
packages/ui	Neutral reusable components: Button, Card, StanceChip, EvidenceLink, CandidateCard, IssueRow.

The first stance contract should support:

```
type CandidateStance = | "SUPPORTS" | "OPPOSES" | "DIFFERENT_APPROACH" | "NO_PUBLIC_POSITION" |
"DECLINED_TO_STATE";
```

Do not create a numeric ideology score or a candidate recommendation field anywhere in the schema.

7. Issue definitions

Create one versioned JSON or YAML file per issue. The coding agent should transcribe the canonical questions from the initial technical handoff rather than inventing or rewriting them. Each file needs at least:

```
{ "id": "housing-supply", "version": 1, "title": "Housing Supply", "billName": "Build America Homes Act",
"goal": "...", "question": "...", "electionCycle": 2026, "status": "active" }
```

• Validate all issue files at build/test time.

• Never silently change an active question. A material wording change requires a new version.

• Do not label issue files Republican, Democratic, left, right, or middle in voter-facing metadata.

• Use the neutral subject categories defined in the initial handoff.

8. Database foundation

Set up PostgreSQL. The first schema should create the entity foundation from the initial handoff, even if many tables remain unused during this setup pass.

Group	Tables / models
Election data	Election, Race, Candidate, RaceCandidate, CandidateAccount
Policy	Issue, IssueVersion
Evidence	Source, SourcePassage, Evidence
Positions	Stance, StanceEvidence
Editorial	CampaignResponse, CorrectionCase, AuditEvent
Automation	ResearchJob

Important database constraints:

• Candidate party is metadata only. No classification routine accepts party as an input needed to determine stance.

• Published Supports, Opposes, or Different Approach records must ultimately be linked to approved evidence.

• No Public Position Found must eventually require a completed research record, not a guess.

• Stance history must be versionable rather than overwritten.

9. Python worker skeleton

Create apps/worker with a modern Python project structure. Use a pyproject.toml rather than a loose requirements.txt if practical. The worker does not need to scrape candidates yet.

```
apps/worker/ pyproject.toml src/ where_they_stand_worker/ __init__.py main.py jobs/ research/ common/
tests/
```

• Add HTTP client, Pydantic, PostgreSQL driver/ORM, Redis client, and test dependencies.

• Create a health-check command or minimal worker entry point.

• Do not add API keys to source control.

• Do not implement unauthorized social scraping.

10. Collector interfaces

Create empty adapter packages or interface skeletons for FEC, Congress.gov, Google Civic, campaign websites, X, Meta, YouTube, and Bluesky. They should compile/import, but they do not need production collection logic in this pass.

The interface should conceptually support discover, fetch, normalize, and health_check. Keep platform-specific code isolated.

11. Local infrastructure

Create infra/docker/docker-compose.yml with PostgreSQL and Redis for local development. Do not place production credentials in this file.

```
services: postgres: image: postgres:17 environment: POSTGRES_DB: where_they_stand POSTGRES_USER: wts
POSTGRES_PASSWORD: localdevelopment ports: - "5432:5432" volumes: -
postgres_data:/var/lib/postgresql/data redis: image: redis:7 ports: - "6379:6379" volumes: postgres_data:
```

12. Environment template

Create .env.example only. Never commit a populated .env.

```
DATABASE_URL=postgresql://wts:localdevelopment@localhost:5432/where_they_stand
REDIS_URL=redis://localhost:6379 FEC_API_KEY= CONGRESS_API_KEY= GOOGLE_CIVIC_API_KEY= X_API_KEY=
META_APP_ID= META_APP_SECRET= YOUTUBE_API_KEY= AI_API_KEY= OBJECT_STORAGE_BUCKET= OBJECT_STORAGE_REGION=
OBJECT_STORAGE_ACCESS_KEY= OBJECT_STORAGE_SECRET_KEY=
```

13. Documentation additions

Preserve the initial handoff and add these text documents as placeholders or concise first drafts:

```
docs/ methodology/ neutrality.md classification.md source-hierarchy.md issue-versioning.md editorial/
corrections.md campaign-responses.md conflicts.md api/ README.md runbooks/ source-outage.md
candidate-dispute.md election-day.md
```

The neutrality document should explicitly say that party affiliation cannot establish a stance and that lack of evidence is not opposition.

14. Neutrality tests

Create tests/neutrality and make neutrality a CI requirement from the beginning.

Test	Expected invariant
Party swap	Changing only party metadata does not change stance output.
Silence	No qualifying evidence never becomes Opposes.
Question version	Candidate comparisons use the same issue version.
Visual parity	Supports and Opposes use equal component size and emphasis.
Candidate ordering	Ordering follows disclosed ballot/alphabetical rules, not party preference.

15. GitHub Actions

Create .github/workflows/ci.yml. On pull requests and pushes to main, run the available checks for both stacks.

• Install Node and pnpm.

• pnpm install with lockfile enforcement.

• lint.

• typecheck.

• tests, including neutrality tests.

• build the web app.

• Install Python.

• Run Python lint/type/test checks.

• Do not require external API keys for ordinary CI.

16. README changes

Do not replace the README the owner already created. Append a Developer Setup section only after the repository commands are known to work.

The setup section should eventually be no more complicated than:

```
git clone cd where-they-stand corepack enable pnpm install docker compose -f
infra/docker/docker-compose.yml up -d cp .env.example .env pnpm dev
```

If the worker requires a second command, document it immediately below. Verify every README command before committing it.

17. What not to build yet

• Do not bulk scrape all candidates.

• Do not classify real candidates.

• Do not send campaign emails.

• Do not implement voter accounts.

• Do not persist voter political answers.

• Do not create recommendation language such as Best Candidate.

• Do not add advertising, donations, or campaign targeting.

• Do not spend time on a finished visual identity.

• Do not create a second repository.

• Do not delete the initial PDF handoff.

18. First coding milestone

The setup pass is complete when a fresh developer can clone the repository and do the following without manual reconstruction:

• Install workspace dependencies.

• Start PostgreSQL and Redis locally.

• Run the Next.js app.

• See the neutral Where They Stand homepage.

• Run TypeScript checks and tests.

• Run Python checks and worker health test.

• Validate all 15 issue-definition files.

• Run neutrality tests.

• Build successfully in CI.

• Read both handoff documents from docs/.

19. Recommended implementation commits

Commit	Scope
chore: initialize workspace	Root pnpm/Turbo config, editor config, gitignore updates.
feat: scaffold web app	Next.js app and neutral shell.
feat: add shared packages	contracts, ui, issue definitions, db package.
feat: add data schema	initial PostgreSQL ORM schema and migration.
feat: scaffold research worker	Python package and health test.
feat: add collector interfaces	source adapter skeletons.
chore: add local infrastructure	Docker Compose and environment template.
test: add neutrality invariants	neutrality test suite.
ci: add repository checks	GitHub Actions.
docs: add developer setup	README developer section and methodology placeholders.

20. Handoff instruction to coding agent

Read the existing repository first. Treat the initial technical handoff as the product specification and this document as the repository-change specification. Preserve existing work. Build the foundation, prove it runs, prove neutrality invariants pass, and stop before production candidate research begins.

When a choice is not specified, prefer the smallest maintainable implementation that keeps the architecture open for the national candidate research system described in the initial handoff.